

# Ensuring Code Integrity in the Era of AI-Assisted Software Development

Arthur Pilone

University of São Paulo

arthurpilone@ime.usp.br

Supervised by Paulo Meirelles and Fabio Kon

**Abstract**—Generative artificial intelligence (GenAI) has taken software engineering by storm, as large language models (LLMs) have quickly come to generate large volumes of human-like text and code. Recent work has shown that, although excellent at mimicking human writing, these models have challenging limitations when writing code. Authors have identified how the code generated can be longer, more complex, worse performing, and contain more serious vulnerabilities than that written by humans. As the use of GenAI now shifts from a bleeding edge novelty to an amenity consolidated as the new status quo for software engineering, a pressing concern arises: It is still unclear how AI-written code impacts the long-term maintenance and evolution of software codebases. This PhD project investigates the previous concern by studying how AI-written code impacts four key qualities of source code: the system’s ability to fulfill functional and non-functional requirements, the architectural cohesion, and the comprehensibility of the source code. By interacting with practitioners, analyzing software repositories, and proposing new approaches to using LLMs for software engineering, we expect to develop the theory and basis necessary to balance the benefits in productivity from GenAI with the ever-pressing needs for long-term software maintenance and evolution.

**Index Terms**—Code Integrity, Code Quality, Generative Artificial Intelligence, Code Comprehension, Software Maintenance

## I. INTRODUCTION

The emergence of generative artificial intelligence (GenAI) and large language models (LLMs) has fundamentally re-defined the way software is developed. The applications of the new technology cover most of the software development workflow, from explaining, creating, and prioritizing requirements [1], [2] to designing and incrementing test cases [3]–[7], supporting automatic program repair [8]–[12], and writing summaries and reviews of code changes [13]–[15]. However, the most sought-after use case for this technology is automated code generation [16]–[20], with company heads boasting plans to replace huge chunks of their software engineering workforce with generative models in the near future.<sup>1 2</sup>

While some authors have attested to the technology’s precision on focused coding tasks, which seem comparable or superior to that of the average programmer [21], recent work points to recurring problems in the quality of the code generated by LLMs.

<sup>1</sup>See Jensen Huang and the ‘death of coding’

<sup>2</sup>See Zuckerberg to ‘move out low-performers faster’

Multiple authors have observed how the code these models generate is often more complex [21], [22], less efficient [23], [24], and more vulnerable [22], [25], [26] than human-written code. Previous work also indicates the models’ incapability of fixing code flaws, having shown the models occasionally make repeated useless changes to the code fragment, only increasing its complexity [25], [27]. From a legal and ethical standpoint, it is still unclear whether most commercially available LLMs infringe the licenses of the publicly available source code collected and used to train their models [28], [29].

Most of these threats are still relevant to code written by humans. However, the speed at which LLM-written code might flood a given codebase usually circumvents safety and quality measures established and consolidated by modern software engineering practices, as the larger code throughput forces development teams to redefine their workflows. Moreover, another serious factor amplifies the threats of carelessly using GenAI for writing source code: the impracticality of distinguishing human from AI-written code [30], [31] or, even worse still, from code written by a single model in particular [32].

As automated code generation suddenly shifted from a far-fetched dream to a crashing reality, research has yet to catch up on the consequences of the large-scale integration of LLM-generated content into software codebases. **It is still unclear what long-term impacts the extensive integration of AI-written code has on software maintenance and evolution.**

## II. CODE INTEGRITY

Intuition hints at possible negative impacts. Mistakenly implemented tests and features, coupled with vulnerable code fragments, could directly affect system functions [26]. Inadequately planned modifications could degrade the software architecture and performance [24]. Changes disregarding system architecture impact how developers comprehend its source code [33]. However, we seek to provide an empirically based answer to one overreaching research question: **RQ<sub>0</sub>: What are the long-term impacts of AI-written code on software maintenance and evolution?**

Reflecting on the most commonly reported flaws in LLM-written code, we identify a set of software qualities affected by such flaws. This PhD project focuses on four categories of source code **integrity**, each corresponding to a quality paramount to the implementation, evolution, maintenance, and well-functioning of any software system.

Accordingly, we split our efforts toward answering (**RQ<sub>0</sub>**) into four research questions investigating the impact of GenAI-written code on each of the four categories of integrity:

**Functional Integrity** is the most elementary of our four qualities. A software system that does not satisfy its functional requirements is, by definition, flawed or incomplete. As such, we ask **RQ<sub>1</sub>: How does AI-written code impact the accomplishment of functional requirements?** Developers should beware not to introduce LLM-written code suggestions when there is a clear hallucination or misunderstanding of the requirements of a given function, class, or module.

**Structural Integrity**, similarly, concerns to non-functional requirements. Functional requirements aside, a codebase that does not support a project’s non-functional requirements is also flawed. Much like a building or any other physical product of engineering, a software system must have *structural integrity* and perform its functions while resisting fractures, deformations, and varying amounts of stress. We ask **RQ<sub>2</sub>: How does AI-written code impact the accomplishment of non-functional requirements?** The code introduced with GenAI should not introduce points of failure, affect code plasticity, or cripple system performance.

**Architectural Integrity** of source code is challenging to maintain with human-written code and is an even bigger concern for LLM-written code due to the models’ context window constraints. Naturally, we ask **RQ<sub>3</sub>: How does AI-written code impact software architecture decay?** A new class that does not respect existing interfaces and design patterns introduces technical debt, hinders software evolution, and may even impact performance. Teams using GenAI should ensure the models and tools they interact with do not violate the design decisions that guide the implementation of the system.

**Comprehension Integrity** is of utmost relevance to software maintenance. As previous work has shown developers spend more time understanding code than writing code itself [33]–[35], a codebase that is excessively complex or unnatural to parse is harder to evolve and maintain. Accordingly, we ask **RQ<sub>4</sub>: How does AI-written code impact how developers comprehend source code?** The excessive complexity and verbosity typical of GenAI outputs should not impede the developers’ ability to comprehend source code.

### III. RESEARCH FOCUSES

Our primary focus is to build an empirically grounded theory on the prolonged impacts of AI-written code on software integrity. Our theory should guide both academic research and industry practice. Based on multivocal literature reviews [36], Grounded Theory principles [37], interviews with practitioners [38], and participant observation [39], [40], we will build, refine and test a theory on how software engineers note and deal with the long-term impact of GenAI on code integrity.

We plan to conduct multiple rounds of interviews to capture how the developers’ perspectives shift as their systems evolve and adapt our theory iteratively.

The participant observation sessions are essential for capturing non-verbalized tendencies as the developers subconsciously adapt their way of work to the impacts of GenAI.

We will also investigate the emergence of “invisible technical debt”, design or quality compromises that accumulate over time, and as organizational factors like policies, guidelines, and team practices influence code review, acceptance, and knowledge sharing as reliance on AI increases.

To complement the qualitative facet of our theory with a quantitative foundation, we will resort to mining software repositories. We will collect data on how LLM-written code affects software complexity, security, and bug fix time, reflecting the code’s structural and architectural integrity. The quantitative data will further guide the refinement of our theory. When possible, we expect to observe how these attributes change before and after the mass popularization of GenAI.

Furthermore, we expect to propose a framework to guide software engineers in leveraging the ever-growing potential of LLM-written code while mitigating its long-term downsides for structural and architectural integrity.

### IV. CURRENT WORK

Although our future work centers on the impacts of GenAI on source code integrity, our current work has explored LLMs’ potential in an approach that supports comprehension integrity. We have designed and evaluated a novel approach that uses text embeddings to augment the information found in GitHub issues with informative user-written app reviews [41]. Our extensive evaluation has demonstrated the potential of the approach to overcome technical and non-technical language differences between users and developers, and its straightforward design provides a foundation for other mechanisms to support comprehension integrity with LLMs and text embeddings.

### V. CONCLUSION

Our study will shed light on a growing concern in the field of software maintenance and evolution: the long-term impacts of the widespread adoption of GenAI code on source code integrity. We will mine data from publicly available software repositories, interact with industry practitioners, and experiment with novel LLM-based mechanisms to create both a theoretical framework and a practical workflow aiming to bring balance between code integrity and the increase in productivity commonly associated with GenAI.

Although, as an early-stage proposal, our specific research focuses and methods are subject to change, we are certain of the relevance of the problem we choose to tackle. We expect our work to leave direct contributions to both academia and practitioners. While the quantitative data we will collect and the framework we will propose should be of immediate value to software developers, our theory should serve as a basis for future work exploring the long-term impacts of GenAI on software maintenance and evolution.

## REFERENCES

- [1] A. Pilone, P. Meirelles, F. Kon, and W. Maalej, "Multilingual crowd-based requirements engineering using large language models," in *Proceedings of the Brazilian Symposium on Software Engineering (SBES)*. SBC, 2024, pp. 679–685.
- [2] S. Schwedt and T. Ströder, "From bugs to benefits: Improving user stories by leveraging crowd knowledge with cruise-ac," 2025. [Online]. Available: <https://arxiv.org/abs/2501.15181>
- [3] P. Alian, N. Nashid, M. Shahbandeh, T. Shabani, and A. Mesbah, "Feature-driven end-to-end test generation," 2025. [Online]. Available: <https://arxiv.org/abs/2408.01894>
- [4] X. Cheng, F. Sang, Y. Zhai, X. Zhang, and T. Kim, "RUG: Turbo LLM for Rust Unit Test Generation," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 634–634. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00097>
- [5] S. Wang, Y. Yu, R. Feldt, and D. Parthasarathy, "Automating a complete software test process using llms: An automotive case study," 2025. [Online]. Available: <https://arxiv.org/abs/2502.04008>
- [6] J. Zhang, Y. Liu, P. Nie, J. J. Li, and M. Gligoric, "exlong: Generating exceptional behavior tests with large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2405.14619>
- [7] K. Zhang, S. Wang, J. Han, X. Zhu, X. Li, S. Wang, and S. Wen, "Your fix is my exploit: Enabling comprehensive dl library api fuzzing with large language models," 2025. [Online]. Available: <https://arxiv.org/abs/2501.04312>
- [8] I. Bouzenia, P. Devanbu, and M. Pradel, "Repairagent: An autonomous, llm-based agent for program repair," 2024. [Online]. Available: <https://arxiv.org/abs/2403.17134>
- [9] P. Deligiannis, A. Lal, N. Mehrotra, R. Poddar, and A. Rastogi, "Rustassistant: Using llms to fix compilation errors in rust code," in *47th International Conference on Software Engineering (ICSE)*, April 2025. [Online]. Available: <https://www.microsoft.com/en-us/research/publication/rustassistant-using-llms-to-fix-compilation-errors-in-rust-code/>
- [10] K. Huang, J. Zhang, X. Meng, and Y. Liu, "Template-Guided Program Repair in the Era of Large Language Models," in *2025 IEEE/ACM 47th International Conference on Software Engineering (ICSE)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 367–379. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/ICSE55347.2025.00030>
- [11] N. Parasaram, H. Yan, B. Yang, Z. Flahy, A. Qudsi, D. Ziaber, E. Barr, and S. Mehtaev, "The fact selection problem in llm-based program repair," 2024. [Online]. Available: <https://arxiv.org/abs/2404.05520>
- [12] A. Z. H. Yang, S. Kolak, V. J. Hellendoorn, R. Martins, and C. L. Goues, "Revisiting unnaturalness for automated program repair in the era of large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2404.15236>
- [13] T. Ahmed and P. Devanbu, "Few-shot training llms for project-specific code-summarization," in *Proceedings of the 37th International Conference on Automated Software Engineering (ASE)*, ser. ASE '22. ACM, 2023. [Online]. Available: <https://doi.org/10.1145/3551349.3559555>
- [14] N. Davila, J. Melegati, and I. Wiese, "Tales from the trenches: Expectations and challenges from practice for code review in the generative ai era," *IEEE Software*, vol. 41, no. 6, pp. 38–45, 2024.
- [15] R. Tufano, S. Masiero, A. Mastropaolo, L. Pascarella, D. Poshyvanyk, and G. Bavota, "Using pre-trained models to boost code review automation," in *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. New York, NY, USA: ACM, 2022, p. 2291–2302. [Online]. Available: <https://doi.org/10.1145/3510003.3510621>
- [16] M. Yuan, J. Chen, Z. Xing, A. Quigley, Y. Luo, T. Luo, G. Mohammadi, Q. Lu, and L. Zhu, "Designrepair: Dual-stream design guideline-aware frontend repair with large language models," 2024. [Online]. Available: <https://arxiv.org/abs/2411.01606>
- [17] Z. Tian, J. Chen, and X. Zhang, "Fixing large language models' specification misunderstanding for better code generation," 2024. [Online]. Available: <https://arxiv.org/abs/2309.16120>
- [18] Y. Zhang, Y. Xie, S. Li, K. Liu, C. Wang, Z. Jia, X. Huang, J. Song, C. Luo, Z. Zheng, R. Xu, Y. Liu, S. Zheng, and X. Liao, "Unseen horizons: Unveiling the real capability of llm code generation beyond the familiar," 2025. [Online]. Available: <https://arxiv.org/abs/2412.08109>
- [19] Y. Wang, Y. Wang, D. Guo, J. Chen, R. Zhang, Y. Ma, and Z. Zheng, "RlCoder: Reinforcement learning for repository-level code completion," 2024. [Online]. Available: <https://arxiv.org/abs/2407.19487>
- [20] X. Jiang, Y. Dong, Y. Tao, H. Liu, Z. Jin, W. Jiao, and G. Li, "Rocode: Integrating backtracking mechanism and program analysis in large language models for code generation," 2025. [Online]. Available: <https://arxiv.org/abs/2411.07112>
- [21] A. Patel, K. Z. Sultana, and B. K. Samanthula, "A comparative analysis between ai generated code and human written code: A preliminary study," in *2024 IEEE International Conference on Big Data (BigData)*, 2024, pp. 7521–7529.
- [22] Z. Liu, Y. Tang, X. Luo, Y. Zhou, and L. F. Zhang, "No need to lift a finger anymore? assessing the quality of code generation by chatgpt," *IEEE Transactions on Software Engineering*, vol. 50, no. 6, pp. 1548–1584, 2024.
- [23] D. Erhabor, S. Udayashankar, M. Nagappan, and S. Al-Kiswani, "Measuring the runtime performance of c++ code written by humans using github copilot," 2024. [Online]. Available: <https://arxiv.org/abs/2305.06439>
- [24] S. Li, Y. Cheng, J. Chen, J. Xuan, S. He, and W. Shang, "Assessing the performance of ai-generated code: A case study on github copilot," in *2024 IEEE 35th International Symposium on Software Reliability Engineering (ISSRE)*, 2024, pp. 216–227.
- [25] R. Khoury, A. R. Avila, J. Brunelle, and B. M. Camara, "How secure is code generated by chatgpt?" in *2023 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2023, pp. 2445–2451.
- [26] N. Tihanyi, T. Bisztray, M. A. Ferrag, R. Jain, and L. C. Cordeiro, "How secure is ai-generated code: a large-scale comparison of large language models," *Empirical Software Engineering*, vol. 30, no. 2, pp. 1–42, 2025.
- [27] B. Johnson and T. Menzies, "Ai over-hype: A dangerous threat (and how to fix it)," *IEEE Software*, vol. 41, no. 6, pp. 131–138, 2024.
- [28] G. M. Kapitsaki, "Generative ai for code generation: Software reuse implications," in *Reuse and Software Quality*, A. Achilleos, L. Fuentes, and G. A. Papadopoulos, Eds. Cham: Springer Nature Switzerland, 2024, pp. 37–47.
- [29] W. Xu, K. Gao, H. He, and M. Zhou, "Licoeval: Evaluating llms on license compliance in code generation," 2025. [Online]. Available: <https://arxiv.org/abs/2408.02487>
- [30] H. Suh, M. Tafreshipour, J. Li, A. Bhattiprolu, and I. Ahmed, "An empirical study on automatically detecting ai-generated source code: How far are we?" 2024. [Online]. Available: <https://arxiv.org/abs/2411.04299>
- [31] Y. Shi, H. Zhang, C. Wan, and X. Gu, "Between lines of code: Unraveling the distinct patterns of machine and human programmers," 2024. [Online]. Available: <https://arxiv.org/abs/2401.06461>
- [32] W. Ma, Y. Song, M. Xue, S. Wen, and Y. Xiang, "The 'code' of ethics: A holistic audit of ai code generators," *IEEE Transactions on Dependable and Secure Computing*, vol. 21, no. 5, pp. 4997–5013, 2024.
- [33] V. Singh, L. L. Pollock, W. Snipes, and N. A. Kraft, "A case study of program comprehension effort and technical debt estimations," in *Proceedings of the 24th International Conference on Program Comprehension (ICPC)*. IEEE, 2016, pp. 1–9.
- [34] A. N. Meyer, E. T. Barr, C. Bird, and T. Zimmermann, "Today was a good day: The daily life of software developers," *Transactions on Software Engineering*, vol. 47, no. 5, pp. 863–880, 2021.
- [35] R. Minelli, A. Mocci, and M. Lanza, "I know what you did last summer - an investigation of how developers spend their time," in *Proceedings of the 23rd International Conference on Program Comprehension (ICPC)*. IEEE, 2015, pp. 25–35.
- [36] V. Geruslu, M. Felderer, and M. Mantyla, "Guidelines for including grey literature and conducting multivocal literature reviews in software engineering," *Information and Software Technology*, vol. 106, 09 2018.
- [37] B. Glaser and A. Strauss, *Discovery of grounded theory: Strategies for qualitative research*. Routledge, 2017.
- [38] W. Edmonds and T. Kennedy, *An Applied Guide to Research Designs: Quantitative, Qualitative, and Mixed Methods*, 01 2017.
- [39] H. Rothstein, A. J. Sutton, and M. Borenstein, *Publication bias in meta-analysis: prevention, assessment and adjustments*. Wiley, 2005.
- [40] C. Robson and K. McCartan, *Real world research: a resource for users of social research methods in applied settings*, 2016.
- [41] A. Pilone, M. Raglianti, M. Lanza, F. Kon, and P. Meirelles, "Automatically augmenting github issues with informative user reviews," 2025, Accepted at ICSME 2025. Preprint. [Online]. Available: <https://linux.ime.usp.br/~arpilone/pubs/icsme25.pdf>